

MicroK8s

1. Desplegar un Kubernetes amb Microk8s , S.O : Linux Ubuntu

Primer que tot, he aprofitat que tinc Vagrant i ja li he donat cert ús per la pràctica de Docker2 per fer-me una màquina virtual d'Ubuntu, ja que jo utilitzo Arch.

```
• ▶ vagrant ssh
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-173-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro
```

1.1. Instal·lació de MicroK8s

Instal·lem MicroK8s amb snap, justament perquè és el canal de distribució oficial. Instal·lo amb exactament la comanda que es dona a l'enunciat per assegurar que utilitzo la versió requerida i amb --classic per desactivar el sandbox estricte de Snap, ja que MicroK8s necessita accés a arxius del sistema:

```
vagrant@microk8s-pedragosa:~$ sudo snap install microk8s --classic --channel=1.33
microk8s (1.33/stable) v1.33.9 from Canonical✓ installed
```

1.2. Uneix-te al grup

```
vagrant@microk8s-pedragosa:~$ sudo usermod -a -G microk8s vagrant
vagrant@microk8s-pedragosa:~$ mkdir -p ~/.kube
vagrant@microk8s-pedragosa:~$ chmod 0700 ~/.kube
```

usermod afegeix el meu usuari (vagrant) al grup microk8s per poder executar les ordres de MicroK8s sense sudo.

Després creem la carpeta .kube al home, que és on K8s guarda la configuració i credencials; i restringim els permissos de la carpeta perquè només pugui accedir l'usuari propietari, ja que les dades poden ser sensibles.

1.3. Comprovar l'estat

```
vagrant@microk8s-pedragosa:~$ microk8s status --wait-ready
microk8s is running
high-availability: no
  datastore master nodes: 127.0.0.1:19001
  datastore standby nodes: none
addons:
  enabled:
    dns                # (core) CoreDNS
    ha-cluster         # (core) Configure high availability on the current node
    helm               # (core) Helm - the package manager for Kubernetes
    helm3              # (core) Helm 3 - the package manager for Kubernetes
  disabled:
    cert-manager       # (core) Cloud native certificate management
    cis-hardening      # (core) Apply CIS K8s hardening
    community           # (core) The community addons repository
    dashboard          # (core) The Kubernetes dashboard
    gpu                # (core) Alias to nvidia add-on
    host-access         # (core) Allow Pods connecting to Host services smoothly
    hostpath-storage    # (core) Storage class; allocates storage from host directory
    ingress             # (core) Ingress controller for external access
    kube-ovn            # (core) An advanced network fabric for Kubernetes
    mayastor           # (core) OpenEBS MayaStor
    metallb             # (core) Loadbalancer for your Kubernetes cluster
    metrics-server      # (core) K8s Metrics Server for API access to service metrics
    minio               # (core) MinIO object storage
    nvidia              # (core) NVIDIA hardware (GPU and network) support
    observability       # (core) A lightweight observability stack for logs, traces and metrics
    prometheus          # (core) Prometheus operator for monitoring and logging
    rbac                # (core) Role-Based Access Control for authorisation
    registry            # (core) Private image registry exposed on localhost:32000
    rook-ceph           # (core) Distributed Ceph storage using Rook
    storage              # (core) Alias to hostpath-storage add-on, deprecated
```

microk8s status mostra l'estat actual de MicroK8s. En aquest cas ens mostra que està en marxa i els addons actius i desactivats. --wait-ready fa que la comanda esperi a que els serveis s'hagin inicialitzat.

1.4. Accés a Kubernetes

```
vagrant@microk8s-pedragosa:~$ microk8s kubectl get nodes
NAME                STATUS    ROLES    AGE     VERSION
microk8s-pedragosa  Ready    <none>   8m58s   v1.33.9
vagrant@microk8s-pedragosa:~$ microk8s kubectl get services
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
kubernetes          ClusterIP   10.152.183.1   <none>       443/TCP    9m4s
vagrant@microk8s-pedragosa:~$ alias kubectl='microk8s kubectl'
vagrant@microk8s-pedragosa:~$ kubectl get nodes
NAME                STATUS    ROLES    AGE     VERSION
microk8s-pedragosa  Ready    <none>   9m45s   v1.33.9
```

microk8s kubectl get nodes mostra els nodes del clúster Kubernetes. De moment només n'hi haurà un (la pròpia màquina). Ha de sortir en estat Ready.

microk8s kubectl get services llista els serveis en funcionament. Per defecte apareix el servei intern kubernetes a la xarxa del clúster.

Per què microk8s kubectl i no simplement kubectl? MicroK8s empaqueta la seva pròpia versió de kubectl per evitar conflictes si ja en tens una d'instal·lada al sistema. L'alias simplement fa que quan escriguis kubectl el sistema executi microk8s kubectl automàticament.

- echo ... >> ~/.bash_aliases: afegeix l'alias al fitxer de configuració perquè persisteixi entre sessions.
- source ~/.bashrc: recarrega la configuració del shell perquè l'alias estigui disponible a la sessió actual sense haver de tancar i reobrir el terminal.

1.5. Implementar una aplicació

```
vagrant@microk8s-pedragosa:~$ microk8s kubectl create deployment nginx --image=nginx
deployment.apps/nginx created
vagrant@microk8s-pedragosa:~$ microk8s kubectl get pods
NAME                READY    STATUS             RESTARTS   AGE
nginx-5869d7778c-gn6k8  0/1      ContainerCreating   0           5s
```

Aquí encara no està «ready».

```
vagrant@microk8s-pedragosa:~$ microk8s kubectl get pods
NAME                READY    STATUS    RESTARTS   AGE
nginx-5869d7778c-gn6k8  1/1      Running   0           62s
```

Aquí ja sí.

microk8s kubectl create deployment nginx --image=nginx crea un *deployment* de Kubernetes, que és la manera estàndard de desplegar una aplicació al clúster.

- `create deployment nginx`: crea un objecte Deployment amb el nom `nginx`. Kubernetes s'encarrega de mantenir-lo en marxa automàticament.
- `--image=nginx`: especifica la imatge de contenidor a utilitzar, en aquest cas la imatge oficial de `nginx` de Docker Hub.

`microk8s kubectl get pods` llista els *Pods* en execució. Un *Pod* és la unitat mínima de Kubernetes: un o més contenidors que s'executen junts al mateix node.

1.6. Utilitza components

```
vagrant@microk8s-pedragosa:~$ microk8s enable dns
Infer repository core for addon dns
Addon core/dns is already enabled
vagrant@microk8s-pedragosa:~$ microk8s enable hostpath-storage
Infer repository core for addon hostpath-storage
Enabling default storage class.
WARNING: Hostpath storage is not suitable for production environments.
        A hostpath volume can grow beyond the size limit set in the volume claim manifest.

deployment.apps/hostpath-provisioner created
storageclass.storage.k8s.io/microk8s-hostpath created
serviceaccount/microk8s-hostpath created
clusterrole.rbac.authorization.k8s.io/microk8s-hostpath created
clusterrolebinding.rbac.authorization.k8s.io/microk8s-hostpath created
Storage will be available soon.
```

`microk8s enable dns` activa el complement CoreDNS, que permet que els serveis i *Pods* es trobin entre ells pel nom en lloc d'haver d'usar IPs directament. Gairebé qualsevol aplicació real el necessita.

`microk8s enable hostpath-storage` activa un proveïdor d'emmagatzematge persistent bàsic que mapeja directoris del node amfitrió als *Pods*. Permet que les aplicacions guardin dades que sobreviuen al reinici del *Pod*.

1.7. Instal·la tres addons: dns, dashboard i ingress (dns l'hem instal·lat al punt anterior)

```
vagrant@microk8s-pedragosa:~$ microk8s enable dashboard
Infer repository core for addon dashboard
Enabling metrics-server
Infer repository core for addon metrics-server
Enabling Metrics-Server
serviceaccount/metrics-server created
clusterrole.rbac.authorization.k8s.io/system:aggregated-metrics-reader created
clusterrole.rbac.authorization.k8s.io/system:metrics-server created
rolebinding.rbac.authorization.k8s.io/metrics-server-auth-reader created
clusterrolebinding.rbac.authorization.k8s.io/metrics-server:system:auth-delegator created
clusterrolebinding.rbac.authorization.k8s.io/system:metrics-server created
service/metrics-server created
deployment.apps/metrics-server created
apiservice.apiregistration.k8s.io/v1beta1.metrics.k8s.io created
clusterrolebinding.rbac.authorization.k8s.io/microk8s-admin created
Metrics-Server is enabled
Infer repository core for addon helm3
Addon core/helm3 is already enabled
Enabling Kubernetes dashboard
Release "kubernetes-dashboard" does not exist. Installing it now.
microk8s enable ingress^[[E
NAME: kubernetes-dashboard
LAST DEPLOYED: Wed May 13 18:44:51 2026
NAMESPACE: kubernetes-dashboard
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
*****
*** PLEASE BE PATIENT: Kubernetes Dashboard may need a few minutes to get up and become ready ***
*****

Congratulations! You have just installed Kubernetes Dashboard in your cluster.

To access Dashboard run:
  kubectl -n kubernetes-dashboard port-forward svc/kubernetes-dashboard-kong-proxy 8443:443

NOTE: In case port-forward command does not work, make sure that kong service name is correct.
      Check the services in Kubernetes Dashboard namespace using:
      kubectl -n kubernetes-dashboard get svc

Dashboard will be available at:
  https://localhost:8443
Applying manifest
secret/microk8s-dashboard-token created

If RBAC is not enabled access the dashboard using the token retrieved with:
```

Instal·lació del dashboard

```
vagrant@microk8s-pedragosa:~$ microk8s enable ingress
Infer repository core for addon ingress
Enabling Ingress
ingressclass.networking.k8s.io/public created
ingressclass.networking.k8s.io/nginx created
namespace/ingress created
serviceaccount/nginx-ingress-microk8s-serviceaccount created
clusterrole.rbac.authorization.k8s.io/nginx-ingress-microk8s-clusterrole created
role.rbac.authorization.k8s.io/nginx-ingress-microk8s-role created
clusterrolebinding.rbac.authorization.k8s.io/nginx-ingress-microk8s created
rolebinding.rbac.authorization.k8s.io/nginx-ingress-microk8s created
configmap/nginx-load-balancer-microk8s-conf created
configmap/nginx-ingress-tcp-microk8s-conf created
configmap/nginx-ingress-udp-microk8s-conf created
daemonset.apps/nginx-ingress-microk8s-controller created
Ingress is enabled
```

Instal·lació d'ingress

microk8s enable dashboard activa la interfície web oficial de Kubernetes, des d'on es pot visualitzar i gestionar tot el clúster gràficament.

microk8s enable ingress activa el controlador d'Ingress, que fa de proxy invers i permet exposar serveis HTTP/HTTPS cap a l'exterior del clúster amb regles de enrutament per nom de domini o ruta.

Cada enable descarrega i desplega els components necessaris al clúster. Un cop acabats, els nous pods apareixen al namespace kube-system.

1.8. Inici i aturada de MicroK8s

```
vagrant@microk8s-pedragosa:~$ microk8s status | head -n 1
microk8s is running
vagrant@microk8s-pedragosa:~$ microk8s stop
Stopped.
Stopped.
vagrant@microk8s-pedragosa:~$ microk8s status | head -n 1
microk8s is not running, try microk8s start
vagrant@microk8s-pedragosa:~$ microk8s start
vagrant@microk8s-pedragosa:~$ microk8s status | head -n 1
microk8s is running
```

microk8s stop atura MicroK8s i tots els seus serveis associats (API server, scheduler, kubelet...). El clúster queda inactiu però la configuració i les dades es conserven. És important executar-ho abans d'apagar la màquina si no vols que MicroK8s arranqui automàticament al proper inici.

microk8s start torna a posar en marxa tots els serveis. L'estat del clúster es recupera tal com estava abans d'aturar-lo.

He pipejat el status a head per poder fer una captura de pantalla amb una mida raonable.

2. Crea un cluster amb multipass, de tres nodes, node0, node 1 i node 2

Primer cal instal·lar multipass:

```
vagrant@microk8s-pedragosa:~$ sudo snap install multipass
multipass 1.16.2 from Canonical✓ installed
```

Ara creem els tres nodes, que triguen uns dos minuts cadascun...:

```
<ass launch --name node0 --cpus 2 --memory 2G --disk 10G
launch failed: instance "node0" already exists
vagrant@microk8s-pedragosa:~$ sudo multipass launch --name node1 --cpus 2 --memory >
Starting node1 ||
```

multipass launch crea i arrenca una nova VM Ubuntu.

- --name node0/1/2: nom identificador de cada VM.
- --cpus 2: assigna 2 nuclis de CPU a cada node.
- --memory 2G: assigna 2 GB de RAM, el mínim recomanat per MicroK8s.
- --disk 10G: assigna 10 GB de disc per a cada node.

No se m'estan creant correctament els nodes, així que no puc continuar amb la pràctica.